

Connected Vehicle Services — Remote Engine Start/Stop

Low-Level Design (LLD)

1. Why This Isn't a Single Synchronous Call

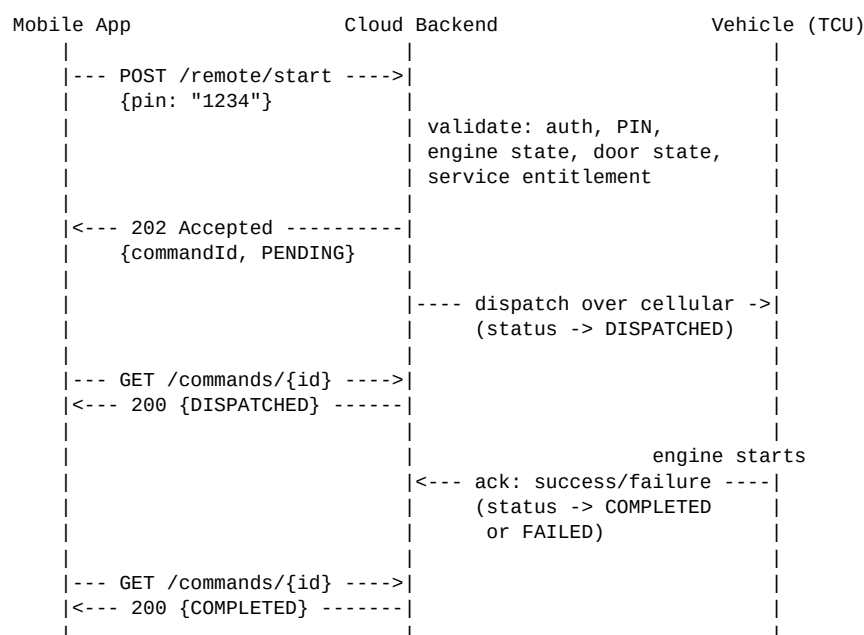
A naive mock would let the mobile app call `POST /remote/start` and get back `{"status": "STARTED"}` immediately. That's not how a real connected-vehicle system works, and testing against that shape would miss entire classes of real bugs (a vehicle that never acknowledges, a command that arrives after the user already left, a race between two commands).

A real system is **asynchronous** across three distinct actors:

1. **Mobile app** — the client, holds the user's intent.
2. **Cloud backend** — validates the request, brokers it to the vehicle, is the system of record for command status.
3. **Vehicle (TCU — Telematics Control Unit)** — the thing that actually has to be reached over a cellular network and physically act.

The cloud cannot make the HTTP request "wait" for the vehicle, because the vehicle might be in a parking garage with poor signal, mid-handshake, or simply slow. So the contract is: **submit now, get an ID back immediately, poll for the real outcome.**

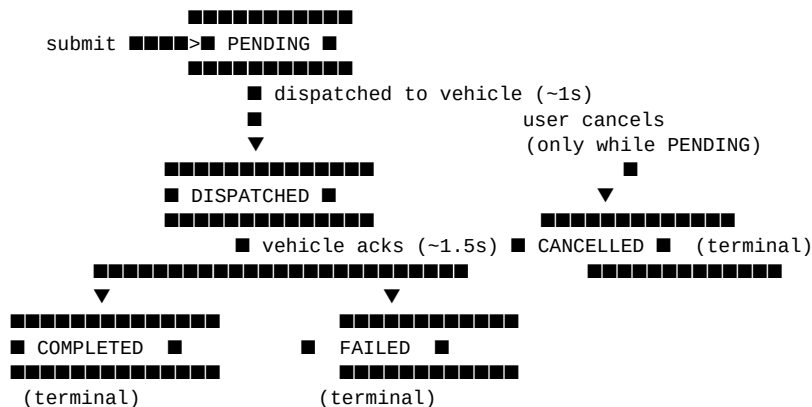
2. Sequence Diagram



```
|--- GET /status ----->|
|<--- 200 {engineOn: true} --|
```

The mobile app is expected to poll `GET /remote/commands/{commandId}` on a short interval (e.g. every 500ms–1s) until status is terminal.

3. Command State Machine



Rules:

- Cancellation is only accepted while `PENDING`. Once `DISPATCHED`, the command has (in the real system) already left the cloud over the air — it cannot be recalled. Attempting to cancel a DISPATCHED (or later) command returns 409 `Conflict`.
- The vehicle's actual state (`engineOn`) only changes at `COMPLETED`. A PENDING or DISPATCHED command has not affected the vehicle yet — this is the detail most naive mocks get wrong, and the one most worth testing.
- `FAILED` carries a `failureReason` string; `COMPLETED` does not.

Mock timing (tunable via `_DISPATCH_DELAY_S` / `_EXECUTE_DELAY_S` in `mock_service.py`): PENDING for ~1s, then DISPATCHED for ~1.5s, then terminal. Total ~2.5s — fast enough for a test's poll loop to complete in a few seconds, slow enough to genuinely observe the intermediate states.

4. API Reference

All endpoints live on the mock at `http://localhost:9000`. All require Authorization: Bearer <token> (any non-empty bearer token — see `_needs_auth()`), returning 401 if missing/malformed.

#	Method	Path	Purpose
1	POST	<code>/api/v1/vehicle/{vin}/remote/start`</code>	Submit an engine-start request. Returns `202` immediately.
2	POST	<code>/api/v1/vehicle/{vin}/remote/stop`</code>	Submit an engine-stop request. Returns `202` immediately.

#	Method	Path	Purpose
3	GET	<code>/api/v1/vehicle/{vin}/remote/commands/{commandId}</code>	Poll a single command's current status.
4	GET	<code>/api/v1/vehicle/{vin}/remote/commands</code>	List all commands issued for a vehicle (history/audit).
5	DELETE	<code>/api/v1/vehicle/{vin}/remote/commands/{commandId}</code>	Cancel a <code>PENDING</code> command. <code>409</code> if already dispatched.
6	GET	<code>/api/v1/vehicle/{vin}/status</code>	Current vehicle state (<code>engineOn</code> , <code>locked</code> , <code>doorsOpen</code> , etc.)
7	POST	<code>/api/v1/_reset</code>	Test-only: reset all vehicle state and command history to defaults.

4.1 `POST /remote/start`

Request body:

Field	Type	Required	Notes
<code>pin</code>	string	yes	Owner's security PIN. Mock accepts <code>"1234"</code> .
<code>duration_minutes</code>	integer	no	Informational only in the mock.
<code>simulateFailure</code>	boolean	no	Test hook — forces the background dispatch to end in <code>FAILED</code> .

Immediate response — 202 Accepted:

```
{ "commandId": "uuid", "vin": "...", "type": "ENGINE_START", "status": "PENDING", "requestedAt": "..." }
```

Immediate rejections (no command created):

Status	Condition
400	<code>pin</code> missing
401	missing/invalid auth
403	incorrect PIN
403	Remote Start service not active on this vehicle
404	unknown VIN
409	engine already running
409	a door is open

4.2 `POST /remote/stop`

Same response shape as start (type: `ENGINE_STOP`). Immediate rejections:

401, 404, and 409 if the engine isn't currently running.

4.3 `GET /remote/commands/{commandId}`

Response — 200:

```
{
  "commandId": "uuid", "vin": "...", "type": "ENGINE_START",
  "status": "PENDING | DISPATCHED | COMPLETED | FAILED | CANCELLED",
  "requestedAt": "...", "dispatchedAt": "... | null",
  "completedAt": "... | null", "failureReason": "... | null"
}
```

404 if the `commandId` doesn't exist for that VIN.

4.4 `DELETE /remote/commands/{commandId}`

200 with `{ "status": "CANCELLED" }` if still `PENDING`.

409 if already `DISPATCHED` or later. 404 if unknown.

4.5 `GET /status`

```
{
  "vin": "...", "locked": true, "engineOn": false, "doorsOpen": false,
  "activeServices": ["REMOTE_START"], "fuelLevelPct": 72, "odometerMiles": 18422
}
```

5. Why Auto-Generation Can't Fully Cover This Flow

AutoBot's ticket-to-test pipeline (see `_generate_ticket_script` in `app.py`) generates one scenario per **single request/response pair** — grounded in a schema of one endpoint at a time. That works cleanly for the immediate-rejection cases in §4.1/§4.2 (missing PIN, bad auth, unknown VIN, etc.) — those are exactly what gets auto-generated when a ticket is tagged `remote_start` or `remote_stop`.

It does **not** work for the polling step (§4.3): a useful test there needs a `commandId` captured from a **previous** call, plus a retry loop — that's a multi-step, stateful test, not a single scenario. Rather than generate something broken, the config documents this limitation explicitly (see the `remote_command_status` / `remote_command_cancel` entries in `config/api_specs.json`), and this LLD points at the real answer:

`tests/examples/remote_start_full_flow.robot` — a hand-written reference test demonstrating the complete chained flow (submit → poll-until-terminal → assert final vehicle state), covering: full success, simulated vehicle failure, cancel-before-dispatch, a full start-then-stop session, and the 409-on-cancel-after-dispatch negative case. Run it directly:

```
python3 mock_service.py & # :9000
python3 -m robot tests/examples/remote_start_full_flow.robot
```

If you want AutoBot itself to generate chained, multi-step tests like this automatically, that's a real extension to the generation pipeline (letting a scenario reference a prior scenario's response), not something the current one-request-per-scenario model supports — flagging it here as a known gap rather than a hidden one.